UNIVERSITÀ DI BOLOGNA



School of Engineering

Master Degree in Automation Engineering

Optimal Control and Reinforcement Learning

Course Project 2 - Flexible Robotic Link

Professor: **Giuseppe Notarstefano**

Students:

Francesco Cenni - 0001194000

Giulio Mazzaroppi - 0001189361

Giacomo Pezzolati - 0001190071

Academic year 2025/2026

Contents

1 Abstract **3**

2 Task 0: Problem Setup and Discretization **5**
2.1 Discretization Methods 5
2.2 Analytic Jacobian Computation 5

3 Task 1: Equilibria and Terminal Cost **6**
3.1 Equilibrium Manifold 6
3.2 Cost Function 6
3.3 Terminal Cost via DARE 6
3.4 Step Size Selection (Armijo Rule) 6

4 Task 2: Trajectory Generation (Newton's Method) **7**
4.1 Reference Generation 7
4.2 Challenges with Standard Newton Method 7
4.3 Dynamic Horizon Extension Strategy 7

5 Task 3: Trajectory Tracking via LQR **8**
5.1 Linear Time-Varying (LTV) LQR 8
5.2 Implementation Details 8

6 Task 4: Trajectory Tracking via MPC **9**
6.1 MPC Formulation 9
6.2 Key Features 9

7 Task 5: Animation **10**
7.1 Visualization 10

8 Conclusion **11**

List of Figures

1 Your descriptive caption goes here. 4

1 Abstract

Introduction



Figure 1: Your descriptive caption goes here.

This report presents the design and implementation of optimal control strategies for a flexible robotic link. The system is modeled as a planar double pendulum where the first joint is actuated, while the second joint is passive and exhibits nonlinear elastic behavior. This structure introduces significant control challenges, specifically:

- **Under-actuation:** The system has two degrees of freedom (DoF) but only one control input (torque at the base).
- **Nonlinear Elasticity:** The stiffness between the links depends nonlinearly on the relative angle θ_2, complicating the dynamics compared to standard linear springs.
- **Non-controllability:** The system is not fully feedback linearizable, and maintaining equilibrium requires precise balancing of gravitational and elastic forces.
- **Low Friction:** The provided parameters (Set 2) involve relatively low friction coefficients, making the system prone to oscillations and sensitive to disturbances.

The project addresses the discretization of dynamics, equilibrium analysis, optimal trajectory generation using a Newton-based method, and trajectory tracking using both Linear Quadratic Regulator (LQR) and Model Predictive Control (MPC).

2 Task 0: Problem Setup and Discretization

The continuous-time dynamics of the double pendulum are governed by the equation:

$$M(q)\ddot{q} + C(q,\dot{q})\dot{q} + F\dot{q} + G(q) = \begin{bmatrix}\tau \\ 0\end{bmatrix} \tag{1}$$

where $q = [\theta_1, \theta_2]^\top$. To utilize numerical optimal control techniques, these dynamics must be discretized.

2.1 Discretization Methods

We implemented two discretization schemes in 'solver.py' to transition from continuous time $\dot{x} = f(x,u)$ to discrete time $x_{k+1} = f_d(x_k, u_k)$:

1. **Forward Euler (FE):** $x_{k+1} = x_k + \Delta t f(x_k, u_k)$. While simple, this method is numerically unstable for stiff systems or large time steps.

2. **Runge-Kutta 4th Order (RK4):** This method computes the state update using a weighted average of four slopes:

$$x_{k+1} = x_k + \frac{\Delta t}{6}(k_1 + 2k_2 + 2k_3 + k_4) \tag{2}$$

   RK4 was selected as the primary method for the project due to its superior accuracy and stability preservation, which is critical for the long-horizon optimizations performed in Task 2.

2.2 Analytic Jacobian Computation

For the Newton-based optimal control algorithm, accurate gradients of the dynamics are required. Relying on finite differences can introduce numerical noise, while symbolic differentiation (via SymPy) proved too slow for iterative solving.

Therefore, we implemented an **analytic Jacobian** computation in 'model.py'. The Jacobians $A = \frac{\partial f}{\partial x}$ and $B = \frac{\partial f}{\partial u}$ were derived manually and implemented using optimized NumPy operations (and Numba JIT compilation for speed). This approach ensures that the linearization required for the Backward Pass in Newton's method is both precise and computationally efficient.

3 Task 1: Equilibria and Terminal Cost

3.1 Equilibrium Manifold

The system's equilibria are defined by states where $\dot{q} = \ddot{q} = 0$. This implies the gravity and stiffness torques must be balanced by the input τ. We analyzed the equilibrium manifold by fixing one generalized coordinate (θ_1 or θ_2) and solving for the other.

- **Elbow-Up vs. Elbow-Down:** We identified distinct configurations such as "Elbow-Up" (unstable) and "Elbow-Down" (stable).
- **Input Requirements:** Due to the passive spring, maintaining certain angles θ_2 requires a non-zero steady-state torque τ_{eq} to counteract the spring force $k(\theta_2)$.

Our analysis, visualized via the compute-equilibrium-manifold function, shows the relationship between the link configuration and the required holding torque.

3.2 Cost Function

For the optimal control problem, we defined the quadratic cost function:

$$J = \sum_{t=0}^{T-1} \left(\frac{1}{2} \Delta x_t^\top Q \Delta x_t + \frac{1}{2} \Delta u_t^\top R \Delta u_t \right) + \frac{1}{2} \Delta x_T^\top Q_T \Delta x_T \tag{3}$$

where $\Delta x = x - x_{ref}$ and $\Delta u = u - u_{ref}$. We selected $Q = \text{diag}(1, 1, 1, 1)$ and $R = 10^{-3} I$ to penalize state deviation while allowing for aggressive control actions.

3.3 Terminal Cost via DARE

A critical aspect of the finite-horizon optimization is the terminal cost Q_T. Simply setting $Q_T = Q$ often fails to stabilize the system at the end of the horizon, especially for short durations.

We computed Q_T by solving the **Discrete Algebraic Riccati Equation (DARE)** based on the linearization at the final target equilibrium (x_{eq}, u_{eq}).

$$P_\infty = A^\top P_\infty A - (A^\top P_\infty B)(R + B^\top P_\infty B)^{-1}(B^\top P_\infty A) + Q \tag{4}$$

Setting $Q_T = P_\infty$ provides an approximation of the infinite-horizon cost-to-go. This essentially informs the solver that the trajectory continues optimally after time T, significantly improving convergence and stability for tasks like the "Swing Up".

3.4 Step Size Selection (Armijo Rule)

To ensure global convergence of the Newton method, we implemented a line search using the Armijo rule. The step size α is reduced ($\alpha \leftarrow \beta\alpha$) until the actual cost reduction is a sufficient fraction of the predicted reduction (based on the gradient). This prevents the solver from taking overly aggressive steps that could destabilize the trajectory.

4 Task 2: Trajectory Generation (Newton's Method)

We employed a Newton-type method for optimal control (Differential Dynamic Programming approach) to generate trajectories transitioning between equilibria.

4.1 Reference Generation

The non-convex nature of the problem requires a reasonable initialization. We explored two reference generation methods:

1. **Stepwise:** A naive instantaneous switch from start to goal. This often results in poor convergence due to the large discontinuity.
2. **Polynomial Interpolation (Poly5):** We generated smooth references for positions, velocities, and accelerations using 5th-order polynomials. This provided a kinematic guide that respects boundary conditions, significantly aiding the solver.

4.2 Challenges with Standard Newton Method

Applying the standard Newton method directly to complex maneuvers (e.g., full swing-up over long horizons) revealed several issues:

- **Local Minima:** The solver often became trapped in local minima where the robot performs a partial swing but fails to reach the upright position.
- **Initialization Sensitivity:** If the initial guess is too far from a feasible solution, the quadratic approximation of the cost is inaccurate, leading to divergence.

4.3 Dynamic Horizon Extension Strategy

To overcome the "local minimum trap" athe difficulty of solving long-horizon problems from scratch, we implemented a **Dynamic Horizon Extension** strategy in "trajectory generation newton.py".

The Algorithm:

1. **Start Small:** Solve the optimal control problem for a short time horizon (e.g., $T_{curr} < T_{final}$) where the goal is reachable or the DARE terminal cost effectively guides the system.
2. **Converge:** Run the Newton solver until convergence for this sub-problem.
3. **Extend:** Use the solution $x^*(t)$ from the current phase as the initial guess for an extended horizon $T_{curr} + \Delta T$. The new tail of the guess is padded with the target equilibrium.
4. **Repeat:** Iterate this process until the full task duration is covered and the terminal error is below a tolerance ($\|x_T - x_{target}\| < \epsilon$).

Results: This approach proved robust. By incrementally solving the problem, the solver is always "warm-started" with a feasible trajectory. Tasks that failed with a direct solve (like the "Topside Pump") converged reliably using this extension scheme.

5 Task 3: Trajectory Tracking via LQR

Once an optimal trajectory (x^*, u^*) is generated, we designed a Linear Quadratic Regulator (LQR) to stabilize the system around this trajectory in the presence of disturbances.

5.1 Linear Time-Varying (LTV) LQR

Since the reference trajectory is not a static point, we linearized the system dynamics along the trajectory, resulting in time-varying matrices A_t and B_t:

$$\delta x_{t+1} = A_t \delta x_t + B_t \delta u_t \tag{5}$$

The feedback law is computed as $u_t = u_t^* - K_t(x_t - x_t^*)$, where K_t is obtained by solving the Riccati equation backward in time.

5.2 Implementation Details

- **Perturbations:** We tested robustness by initializing the system with random state perturbations ($\epsilon \sim U[-0.05, 0.05]$).
- **Horizon Extension:** To keep the robot at the goal after the maneuver ends, we extended the LQR controller horizon beyond the trajectory end time, using the steady-state LQR gain for the final equilibrium.

The results (see Figure 1) demonstrate that the LTV-LQR effectively rejects initial disturbances and tracks the complex optimal maneuver.

6 Task 4: Trajectory Tracking via MPC

While LQR is effective for unconstrained tracking, it cannot explicitly handle actuator saturation. We implemented a linear Model Predictive Control (MPC) scheme to address this.

6.1 MPC Formulation

At each time step t, we solve a finite-horizon optimization problem:

$$\begin{aligned} \min_{\Delta u} \quad & \sum_{k=0}^{T_{pred}-1} (\|\Delta x_k\|_Q + \|\Delta u_k\|_R) + \|\Delta x_{T_{pred}}\|_{Q_T} \\ \text{s.t.} \quad & \Delta x_{k+1} = A_{t+k}\Delta x_k + B_{t+k}\Delta u_k \\ & u_{min} \leq u^*_{t+k} + \Delta u_k \leq u_{max} \end{aligned}$$

where T_{pred} is the prediction horizon (set to 50 steps).

6.2 Key Features

- **Constraints:** We imposed hard constraints on the input torque ($u \in [u_{min}, u_{max}]$) to respect physical motor limits, which LQR ignores.
- **Receding Horizon:** Only the first control input is applied, and the optimization is repeated at the next step.
- **Solver:** We utilized CasADi with IPOPT to solve the constrained QP/NLP at each step.

The MPC controller showed superior performance in scenarios where large perturbations would cause an LQR controller to request infeasible torques.

7 Task 5: Animation

7.1 Visualization

An animation suite was developed (‘animation.py‘) to visualize the results. It displays the double pendulum link, the reference trajectory (ghost), and the actual tracked path. This visual verification was crucial for debugging the "elbow-flip" behaviors inherent in the nonlinear stiffness model.

8 Conclusion

This project successfully demonstrated the control of a flexible under-actuated link.

- The **analytic Jacobian** and **RK4 discretization** provided the necessary precision for optimization.
- The **DARE-based terminal cost** significantly improved the stability of finite-horizon approximations.
- The **Dynamic Horizon Extension** strategy was the key enabler for generating complex trajectories, effectively bypassing local minima issues common in nonlinear optimization.
- Finally, **LQR and MPC** proved effective for tracking, with MPC offering the critical advantage of handling actuator constraints.